Exercices sur les tableaux

Exercice 1 :

Écrivez un tableau multidimensionnel associatif dont les clés sont des noms de personne et les valeurs des tableaux indicés contenant le prénom, la ville de résidence et l'âge de la personne.
La fonction `print_r()` affiche la structure du tableau.

Exercice 2 :
Écrivez un tableau multidimensionnel associatif dont les clés sont des noms de personne et les valeurs des tableaux associatifs dont les clés sont le prénom, la ville de résidence et l'âge de la personne avec une série de valeurs associées.
La fonction `print_r()` affiche la structure du tableau.

Exercice 3 :

Utilisez une boucle foreach pour lire les tableaux des exercices 1 et 2.

Exercice 4 :
Créez un tableau contenant une liste d'adresses e-mail. Extrayez le nom de serveur de ces données, puis réalisez des statistiques sur les occurrences de chaque fournisseur d'accès.

Annexe : Les tableaux en PHP

Introduction
Un tableau (aussi appelé **array**) est une variable dans laquelle on peut enregistrer de nombreuses informations. On en distingue deux types :

- les tableaux numérotés ;
- les tableaux associatifs.

1. Les tableaux numérotés

Créer un tableau numéroté

Clé	Valeur
0	François
1	Michel
2	Nicole
3	Véronique
4	Benoît
...	...

Exemple : `$prenoms` est un **array**

Dans un array, les valeurs sont rangées dans des « cases » différentes. Ici, nous travaillons sur un array *numéroté*, c'est-à-dire que chaque case est identifiée par un numéro. Ce numéro est appelé **clé**.

NB: Un array numéroté commence toujours à la case n°0 !

Construire un tableau numéroté :

Pour créer un tableau numéroté en PHP, on utilise généralement la fonction `array`.

```php
<?php
// La fonction array permet de créer un array
$prenoms = array ('François', 'Michel', 'Nicole', 'Véronique', 'Benoît');
?>
```

Il y en d'autres méthodes pour construire un tableau numéroté :

```php
<?php
$prenoms[0] = 'François';
$prenoms[1] = 'Michel';
$prenoms[2] = 'Nicole';
?>
```

Ou bien :

```php
<?php
$prenoms[] = 'François'; // Créera $prenoms[0]
$prenoms[] = 'Michel'; // Créera $prenoms[1]
$prenoms[] = 'Nicole'; // Créera $prenoms[2]
?>
```

Afficher un tableau numéroté :

On peut afficher un tableau numéroté à l'aide d'une boucle.

Afficher avec une boucle for

La boucle for a besoin d'un paramètre d'arrêt. Vous allez employer pour cela la fonction **count()**, qui retourne le nombre total d'éléments de la boucle. L'utilisation de la variable $i comme compteur de la boucle permet de parcourir l'ensemble des valeurs du tableau unidimensionnel.

```php
<?php
$montab=array("Paris","London","Brüssel");
for   ($i=0;$i<count($montab);$i++) {
    echo "L'élément $i est $montab[$i]<br />";
}
?>
```

Afficher avec une boucle while

Pour un tableau à une seule dimension, l'expression booléenne contenue dans l'instruction while est isset($tab[$i]). Cette expression prend la valeur TRUE tant que l'élément désigné par $tab[$i] existe. Sinon, elle prend la valeur FALSE, ce qui est le cas en fin de tableau.

```php
<?php
$montab=array("Paris","London","Brüssel");
$i=0;
while(isset($montab[$i])  ) {
    echo "L'élément $i est $montab[$i]<br />";
    $i++;
}
?>
```

2. Les tableaux associatifs

Les tableaux associatifs fonctionnent sur le même principe, sauf qu'au lieu de numéroter les cases, on va les étiqueter en leur donnant à chacune un nom différent.

Par exemple, supposons que je veuille, dans un seul array, enregistrer les coordonnées de quelqu'un (nom, prénom, adresse, ville, etc.). Si l'array est numéroté, comment savoir que le n°0 est le nom, le n°1 le prénom, le n°2 l'adresse… ? C'est là que les tableaux associatifs deviennent utiles.

Construire un tableau associatif

Pour en créer un, on utilisera la fonction **array** comme tout à l'heure, mais on va mettre « l'étiquette » devant chaque information. La syntaxe de la fonction array() est la suivante :

```php
$tabasso  =  array("cléA"=>valeurA,  "cléB"=>valeurB,…  "cléZ"=>valeurZ)
```

Exemple :

```php
<?php
// On crée notre array $coordonnees
$coordonnees = array (
    'prenom' => 'François',
    'nom' => 'Dupont',
    'adresse' => '3 Rue du Paradis',
    'ville' => 'Marseille');
?>
```

Nous avons créé un tableau qui ressemble à la structure suivante :

Clé	Valeur
prenom	François
Nom	Dupont
adresse	3 Rue du Paradis
Ville	Marseille

Il est aussi possible de créer le tableau case par case, comme ceci :

```php
<?php

$coordonnees['prenom'] = 'François';

$coordonnees['nom'] = 'Dupont';

$coordonnees['adresse'] = '3 Rue du Paradis';

$coordonnees['ville'] = 'Marseille';

?>
```

Dans un tableau associatif, la notion d'ordre des éléments perd la valeur qu'elle peut avoir dans un tableau indicé. Les clés ne sont pas numérotées, par exemple. On peut énumérer clés et valeurs dans un ordre différent sans que cela gène la lecture individuelle de chaque élément.

Afficher un tableau associatif

Pour afficher un élément, il suffit d'indiquer le nom de cet élément entre crochets, ainsi qu'entre guillemets ou apostrophes puisque l'étiquette du tableau associatif est un texte.

Par exemple, pour extraire la ville, on devra taper :

```php
<?php

echo $coordonnees['ville'];

?>
```

Ce code affiche : « Marseille ».

Parcourir un tableau associatif :

Pour les tableaux associatifs on peut utiliser un seul type de boucle plus adapté aux tableaux qu'on n'a pas encore vu jusqu'ici :**`foreach`**. C'est une sorte de boucle `for` spécialisée dans les tableaux.

`Foreach` va passer en revue chaque ligne du tableau, et lors de chaque passage, elle va mettre la valeur de cette ligne dans une variable temporaire (par exemple `$element`).

```php
<?php
$prenoms = array ('François', 'Michel', 'Nicole', 'Véronique', 'Benoît');

foreach($prenoms as $element)
{
    echo $element . '<br />'; // affichera $prenoms[0], $prenoms[1] etc.
}
?>
```

François
Michel
Nicole
Véronique
Benoît

Remarque : On peut utiliser **foreach** pour parcourir un tableau numéroté comme dans l'exemple précédent.

Et pour un tableau associatif cela devient :

```php
<?php
$coordonnees = array (
    'prenom' => 'François',
    'nom' => 'Dupont',
    'adresse' => '3 Rue du Paradis',
    'ville' => 'Marseille');

foreach($coordonnees as $element)
{
    echo $element . '<br />';
}
?>
```

François
Dupont
3 Rue du Paradis
Marseille

`Foreach` va mettre tour à tour dans la variable `$element` le prénom, le nom, l'adresse et la ville contenus dans l'array `$coordonnees`.

On met donc entre parenthèses :

1. d'abord le **nom de l'array** (ici `$coordonnees`) ;
2. ensuite le mot-clé **`as`** (qui signifie « en tant que ») ;
3. enfin, le **nom d'une variable** que vous choisissez et qui va contenir tour à tour chacun des éléments de l'array (ici **`$element`**).

Entre les accolades, on n'utilise donc que la variable `$element`.
La boucle s'arrête lorsqu'on a parcouru tous les éléments de l'array.

On remarque bien **qu'on n'a pas besoin de connaitre le nombre d'éléments** du tableau à l'avance via une boucle foreach.

Toutefois, avec cet exemple, on ne récupère que la valeur. Or, on peut aussi récupérer la clé de l'élément. On doit dans ce cas écrire `foreach` comme ceci :

```php
<?php foreach($coordonnees as $cle => $element) ?>
```

À chaque tour de boucle, on disposera non pas d'une, mais de deux variables :

- **`$cle`**, qui contiendra **la clé** de l'élément en cours d'analyse (« prenom », « nom », etc.) ;
- **`$element`**, qui contiendra la valeur de l'élément en cours (« François », « Dupont », etc.).

Testons le fonctionnement avec un exemple :

php

```php
<?php
$coordonnees = array (
    'prenom' => 'François',
    'nom' => 'Dupont',
    'adresse' => '3 Rue du Paradis',
    'ville' => 'Marseille');

foreach($coordonnees as $cle => $element)
{
    echo '[' . $cle . '] vaut ' . $element . '<br />';
}
?>
```

[prenom] vaut François
[nom] vaut Dupont
[adresse] vaut 3 Rue du Paradis
[ville] vaut Marseille

Récupérer la clé de l'élément

Afficher rapidement un array avec print_r

Di on veut afficher un array on peut utiliser une boucle for ou encore mieux foreach. . Mais si on n'a pas pas besoin d'une mise en forme spéciale et qu'on veut juste savoir ce que contient l'array, on peut faire appel à la fonction **`print_r`**. C'est une sorte de **`echo` spécialisé dans les arrays**.

Cette commande a toutefois un défaut : elle ne renvoie pas de code HTML comme`<br />`pour les retours à la ligne. Pour bien les voir, il faut donc utiliser la balise HTML`<pre>`qui nous permet d'avoir un affichage plus correct.

```php
<?php
$coordonnees = array (
    'prenom' => 'François',
    'nom' => 'Dupont',
    'adresse' => '3 Rue du Paradis',
    'ville' => 'Marseille');

echo '<pre>';
print_r($coordonnees);
echo '</pre>';
?>
```

```
Array
(
    [prenom] => François
    [nom] => Dupont
    [adresse] => 3 Rue du Paradis
    [ville] => Marseille
)
```

Avec utilisation de la balise

Rechercher dans un tableau

Nous allons maintenant faire des recherches dans des arrays. Cela sera parfois très utile pour savoir si l'array contient ou non certaines informations.
Nous allons voir trois types de recherches, basées sur des fonctions PHP :

- **`array_key_exists`** : pour vérifier si une clé existe dans l'array ;
- **`in_array`** : pour vérifier si une valeur existe dans l'array ;
- **`array_search`** : pour récupérer la clé d'une valeur dans l'array.

Vérifier si une clé existe dans l'array : `array_key_exists`

Voici notre problème : on a un array, mais on ne sait pas si la clé qu'on cherche s'y trouve.
Pour vérifier ça, on va utiliser la fonction **`array_key_exists`** qui va parcourir le tableau pour nous et nous dire s'il contient cette clé.

On doit d'abord lui donner le nom de la clé à rechercher, puis le nom de l'array dans lequel on fait la recherche :

```php
<?php array_key_exists('cle', $array); ?>
```

La fonction renvoie un booléen, c'est-à-dire `true` (vrai) si la clé est dans l'array, et `false` (faux) si la clé ne s'y trouve pas. Ça nous permet de faire un test facilement avec un `if` :

php

```php
<?php
$coordonnees = array (
    'prenom' => 'François',
    'nom' => 'Dupont',
    'adresse' => '3 Rue du Paradis',
    'ville' => 'Marseille');

if (array_key_exists('nom', $coordonnees))
{
    echo 'La clé "nom" se trouve dans les coordonnées !';
}

if (array_key_exists('pays', $coordonnees))
{
    echo 'La clé "pays" se trouve dans les coordonnées !';
}

?>
```

La clé "nom" se trouve dans les coordonnées !

Vérifier si la clé est dans l'array $coordonnees

Vérifier si une valeur existe dans l'array : `in_array`

Le principe est le même que `array_key_exists`... mais cette fois on recherche dans les valeurs. **`in_array`** renvoie `true` si la valeur se trouve dans l'array, `false` si elle ne s'y trouve pas.

Pour changer un peu de notre array `$coordonnees`, on va en créer un nouveau (numéroté) composé de fruits.

php

```php
<?php
$fruits = array ('Banane', 'Pomme', 'Poire', 'Cerise', 'Fraise', 'Framboise');

if (in_array('Myrtille', $fruits))
{
    echo 'La valeur "Myrtille" se trouve dans les fruits !';
}

if (in_array('Cerise', $fruits))
{
    echo 'La valeur "Cerise" se trouve dans les fruits !';
}
?>
```

La valeur "Cerise" se trouve dans les fruits !

Vérifier si une valeur existe dans l'array $fruits

Récupérer la clé d'une valeur dans l'array : `array_search`

`array_search` fonctionne comme `in_array` : il travaille sur les valeurs d'un array. Voici ce que renvoie la fonction :

- si elle a trouvé la valeur, `array_search` renvoie la clé correspondante (c'est-à-dire le numéro si c'est un array *numéroté*, ou le nom de la clé si c'est un array *associatif*);
- si elle n'a pas trouvé la valeur, `array_search` renvoie `false`.

On reprend l'array numéroté avec les fruits :

php

```php
<?php
$fruits = array ('Banane', 'Pomme', 'Poire', 'Cerise', 'Fraise', 'Framboise');

$position = array_search('Fraise', $fruits);
echo '"Fraise" se trouve en position ' . $position . '<br />';

$position = array_search('Banane', $fruits);
echo '"Banane" se trouve en position ' . $position;
?>
```

"Fraise" se trouve en position 4
"Banane" se trouve en position 0

Récupérer la clé d'une valeur